A Knowledge-Based Decision Support System for Spotify Context-Aware Music Recommendation

Stefano Colella • Matricola: 865075
Master's Degree in Computer Science • University of Bari Aldo Moro

Abstract: Traditional music recommendation platforms typically rely on past listening history or global popularity, ignoring the listener's real-time activity and physical state. This project presents a rule-based Decision Support System (DSS) written in SWI-Prolog, designed as an intelligent context-aware recommendation engine integrated with a Spotify-style catalog and wearable telemetry.

The system merges two structured JSON datasets (Spotify track audio features and smartwatch user biometrics) into a unified Prolog knowledge base. Using native backward chaining inference, it evaluates track eligibility through three orthogonal criteria: activity context compatibility, study focus protection, and physiological safety barriers. Eligible candidates are ranked by a multi-attribute utility function balancing goal cadence alignment with user genre taste, and recursive list processing generates personalized, safe context playlists.

1. Introduction & Motivation

Conventional streaming recommendation algorithms assume that musical taste is static across all daily situations. In practice, suitability depends heavily on current physical and cognitive context: high-tempo energetic songs are ideal during cardio exercise, whereas they are unsuitable when a user is already experiencing high stress and elevated heart rate. Similarly, vocal tracks can interfere with focus during study sessions, while very energetic music disrupts sleep preparation.

To address these scenarios, this project implements a declarative, transparent Decision Support System in SWI-Prolog. Rather than operating as a black-box statistical predictor, the system explicitly separates domain knowledge (facts) from inference logic (rules), providing deterministic recommendations, ranked context playlists, and clear pass/fail explanations for every decision.

2. System Architecture

The codebase is organized into three decoupled layers:

1. **Knowledge Base Generator (`generate_kb.py`):** Parses the two raw JSON files (`data/spotify_tracks.json` and `data/smartwatch_biometrics.json`) and emits normalized Prolog facts into `knowledge_base.pl` .
2. **Decision Rules & Playlist Engine (`dss_rules.pl`):** Implements backward-chaining rules for user state detection, audio filtering, safety constraints via negation as failure, multi-attribute utility scoring, and recursive playlist assembly.
3. **User Interface & Verification Module (`main.pl`):** Exposes high-level DSS commands for single-track recommendations, context playlists, genre filtering, safety audits, feasibility explanations, and native unit tests.

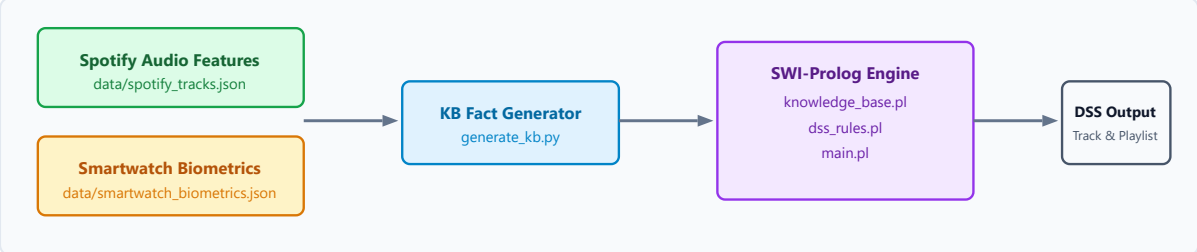


Figure 1: Knowledge-Based DSS pipeline merging track catalog and user context into SWI-Prolog.

3. Knowledge Base Representation

The unified knowledge base (`knowledge_base.pl`) stores 30 tracks and 6 user profiles using two normalized Prolog predicates:

3.1 Spotify Track Catalog: `track/8`

```
track(Id, Title, Artist, Genre, BPM, Valence, Energy, Instrumentalness).
```

- `BPM` : Musical tempo in beats per minute (ranging from 55.0 to 165.0).
- `Valence` (∈ [0.0, 1.0]): Musical positiveness and cheerful tone.
- `Energy` (∈ [0.0, 1.0]): Perceived acoustic intensity and loudness.
- `Instrumentalness` (∈ [0.0, 1.0]): Likelihood that the track has no spoken/sung vocals (≥ 0.70 indicates instrumental).

3.2 User Context & Biometric Profiles: `user/7`

```
user(Id, Name, Activity, HeartRate, StressLevel, TargetGoal, PreferredGenres).
```

- `Activity` : Current user context (`high_stress_work` , `cardio_running` , `deep_focus_study` , `sleep_preparation` , `post_workout_recovery` , `active_commute`).
- `HeartRate` : Live heart rate measurement in BPM.
- `StressLevel` : Estimated stress level on a 0–100 scale.
- `TargetGoal` : Current listening goal (e.g., `stress_downregulation` , `cardio_peak_performance` , `deep_focus_flow` , `sleep_induction`).
- `PreferredGenres` : List of the user's preferred Spotify genres (e.g., `[ambient, classical, lofi]`).

4. Inference Engine & Decision Rules

The reasoning engine in `dss_rules.pl` leverages SWI-Prolog's native backward chaining. Grounding is enforced via `track/8` and `user/7` so that predicates remain bidirectional for both forward recommendations and inverted catalog queries:

```
recommendable_track(TrackId, UserId) :-
    track(TrackId, _, _, _, _, _, _, _),
    user(UserId, _, _, _, _, _, _),
    activity_compatible(TrackId, UserId),
    focus_compatible(TrackId, UserId),
    safe_track(TrackId, UserId).
```

4.1 The Three Orthogonal Constraint Pillars

1. **Activity & Biometric Match (`activity_compatible/2`):** Classifies the user's current state using mutually exclusive cuts (`!`) and selects matching audio constraints:
   - *High-stress user* (`Goal == stress_downregulation` or `HeartRate ≥ 100` and `Stress ≥ 65`): requires `relaxing_track/1` (`BPM ≤ 75` , `Energy ≤ 0.35` , `Instrumentalness ≥ 0.70`).
   - *Workout user* (`Goal == cardio_peak_performance` or `Activity == cardio_running` with `HeartRate ≥ 120`): requires `workout_track/1` (`BPM ≥ 120` , `Energy ≥ 0.70`).
   - *Sleep user* (`Goal == sleep_induction` or `Activity == sleep_preparation`): requires `sleep_track/1` (`BPM ≤ 64` , `Energy ≤ 0.20` , `Instrumentalness ≥ 0.85`).
   - *Normal state* (default fallback via `normal_state_user/1`): requires `moderate_track/1` (`70 ≤ BPM ≤ 115` , `Energy ≤ 0.65` , `Valence ≥ 0.40`).
2. **Focus & Study Protection (`focus_compatible/2`):** When `Activity == deep_focus_study` or `Goal == deep_focus_flow` , enforces `instrumental_focus_track/1` (`Instrumentalness ≥ 0.70` , `70 ≤ BPM ≤ 95` , `Energy ≤ 0.55`) to prevent vocal distractions. For non-study activities, this check succeeds unconditionally.
3. **Safety Constraints (`safe_track/2` via `\+ unsafe_track/2`):** Implements negative safety constraints using negation as failure (`\+`):
   - Rejects tracks with `BPM > 100` or `Energy > 0.60` when `HeartRate > 110` and `Stress > 75` .
   - Rejects tracks with `Energy > 0.50` during `high_stress_work` if `Stress > 80` .

4.2 Recursive Context Playlist Generation

Rather than selecting only one song, a streaming DSS must assemble playlists that strictly obey all user constraints. The predicate `build_playlist(UserId, MaxLength, Playlist)` collects all ranked eligible tracks via `ranked_tracks/2` and recursively constructs a duplicate-free list using `take_top_tracks/4` :

```
build_playlist(UserId, MaxLength, Playlist) :-
    ranked_tracks(UserId, SortedCandidates),
    take_top_tracks(SortedCandidates, MaxLength, [], Playlist).

take_top_tracks([], _, _, []) :- !.
take_top_tracks(_, N, _, []) :- N =< 0, !.
take_top_tracks([_-TId|Rest], N, Seen, [TId|Tail]) :-
    \+ member(TId, Seen), !,
    N1 is N - 1,
    take_top_tracks(Rest, N1, [TId|Seen], Tail).
```

Additionally, `playlist_avg_bpm/2` recursively traverses the resulting track list to compute the average tempo of the generated playlist.

5. Spotify Utility Scoring & Ranking

Eligible candidates are scored on a 0–100 scale by `calculate_utility/3`, combining context alignment with personal Spotify taste:

$$\text{FinalScore} = \min(100.0,\ 0.50 \cdot \text{S}_{\text{tempo}} + \text{Bonus}_{\text{genre}} + \text{S}_{\text{mood}})$$

- **Target Tempo Score (S_{tempo}, max 100 × 0.50 = 50 pts):** Measures proximity to the goal's ideal target BPM (e.g., 65 for stress reduction, 145 for cardio, 80 for focus): `DeltaBPM = abs(BPM - TargetBPM)`, S_{tempo} `= max(20.0, 100.0 - 1.5 · DeltaBPM)`.
- **Spotify Genre Affinity ($Bonus_{genre}$ = 25 pts):** Awarded when the track's genre belongs to the user's `PreferredGenres` list.
- **Mood & Acoustic Fit (S_{mood}, max 25 pts):** S_{mood} `= (Valence · 15.0) + (Instrumentalness · 10.0)`.

6. Logical Verification & Test Results

The native Prolog test suite in `main.pl` (invoked via `?- run_tests.`) verifies 7 formal properties, including positive matching, recursive playlist safety, and negative barrier enforcement:

#	Test Description	Input Profile / Condition	Verified Property	Status
1	High-Stress Relaxation	Sarah Miller (`high_stress_work` , HR 114, Stress 84)	Best track has `BPM ≤ 75` and `Energy ≤ 0.35`	PASS
2	Cardio Workout Cadence	Alex Chen (`cardio_running` , HR 146)	Best track has `BPM ≥ 120` and `Energy ≥ 0.70`	PASS
3	Study Instrumental Focus	Marco Rossi (`deep_focus_study` , HR 74)	Best track has `Instrumentalness ≥ 0.70`	PASS
4	Sleep Preparation Window	David Becker (`sleep_preparation` , HR 68)	Best track has `BPM ≤ 64` and `Energy ≤ 0.20`	PASS
5	Playlist Safety Consistency	`build_playlist(user_sarah, 3, P)`	Every track in playlist satisfies `safe_track/2`	PASS
6	Safety Barrier (Negative)	Sarah Miller with `adrenaline_rush_rock` (150 BPM)	Rejected via `\+ safe_track/2`	PASS
7	Focus Barrier (Negative)	Marco Rossi with `euphoric_run_anthem` (vocal pop)	Rejected via `\+ focus_compatible/2`	PASS

7. Real Execution Walkthroughs

Exact outputs obtained by querying SWI-Prolog on `main.pl`:

Case 1: High-Stress Work & Context Playlist (Sarah Miller)

Profile: HR = 114 BPM, Stress = 84/100, Activity = `high_stress_work`, Preferred Genres = `[ambient, classical, lofi]`.

Single Recommendation: "*Clair de Lune (Debussy)*" by Sinfonia Chambers (Match Score: 90.0/100, classical, 64 BPM, Energy: 0.18, Instrumental: 0.94).

Generated Spotify Context Playlist (Top 3):

```
1. "Clair de Lune (Debussy)" - Sinfonia Chambers    (classical, 64 BPM, E:0.18) [Score: 90.0]
2. "Gymnopedie No. 1 (Satie)" - Aura Piano Quartet  (classical, 66 BPM, E:0.15) [Score: 89.5]
3. "Peaceful Mind Breeze" - Zenith Soundscapes      (ambient,   68 BPM, E:0.22) [Score: 89.0]
Summary: 3 tracks selected | Average Playlist Tempo: 66.0 BPM
```

Case 2: Cardio Running (Alex Chen)

Profile: HR = 146 BPM, Stress = 45/100, Activity = `cardio_running`, Preferred Genres = `[electronic, synthwave, techno, pop]`.

Single Recommendation: "*Subterranean Stamina Loop*" by Klangwerk (Match Score: 89.5/100, techno, 140 BPM, Energy: 0.92, Instrumental: 0.90). Matches Spotify taste (techno).

Case 3: Deep Focus Study (Marco Rossi)

Profile: HR = 74 BPM, Stress = 32/100, Activity = `deep_focus_study`, Preferred Genres = `[lofi, classical, ambient, electronic]`.

Single Recommendation: "*Analog Cassette Memory*" by BeatFoundry (Match Score: 92.3/100, lofi, 80 BPM, Energy: 0.38, Instrumental: 0.86). Matches Spotify taste (lofi) and prevents vocal distractions.

Case 4: Feasibility Check / Why-Not Explanation (`explain_rejection/2`)

```
?- explain_rejection(user_sarah, adrenaline_rush_rock).
  [FAIL] Activity match: incompatible tempo or energy for current activity.
  [PASS] Focus check: compatible with current cognitive mode.
  [FAIL] Safety check: unsafe BPM/energy given high stress or heart rate.
 RESULT: Track is REJECTED.
```

8. Conclusion

By integrating user biometric context and activity constraints directly into a rule-based Prolog engine alongside Spotify audio features and genre preferences, this DSS avoids the pitfalls of context-blind recommendation. Its clean three-pillar design eliminates redundant checks, prevents safety contradictions across both single-track recommendations and recursive playlists, and delivers deterministic, transparent explanations for every decision.